Actual HW3

Sam Olson

Q1

Using the mixture model (1) and (2) fit the data in Data Set 1 and Data Set 2 separately using maximum likelihood. Produce approximate 95% confidence intervals for the modes of the distributions – these are the value of the parameter ξ in (2). Conduct any additional analysis needed to construct a likelihood ratio test of a reduced model in which the parameters ξ and θ are identical for both data sets against a full model in which those parameters differ between the two heights. Graph the extreme value density or densities for whichever model (full or reduced) you select.

Data model:

$$f(y \mid \mu_i) = \frac{1}{(2\pi)^{1/2}} \exp\left[-\frac{1}{2}(y-\mu_i)^2\right]. \tag{1}$$

Mixing distribution:

$$g(\mu \mid \xi, \theta) = \frac{1}{\theta} \exp\left(-\frac{\mu-\xi}{\theta}\right) \exp\left[-\exp\left(-\frac{\mu-\xi}{\theta}\right)\right], \qquad \theta > 0. \tag{2}$$

A:

```
library(tidyverse)
# 50m
m50 <- read.table("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/winddat1.txt", header = T
# 80m
m80 <-  read.table("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/winddat2.txt", header =
```

```
# ----------------------------
# Setup
# ----------------------------
y50 <- as.numeric(m50$y)
y80 <- as.numeric(m80$y)

# ----------------------------
# Model pieces
# f(y|mu) = N(mu, 1)
# g(mu|xi,theta) = right-skew Gumbel density
# m(y|xi,theta) = int f(y|mu) g(mu|xi,theta) dmu
# ----------------------------

f_y_given_mu <- function(y, mu) dnorm(y, mean = mu, sd = 1)
```

```r
g_mu <- function(mu, xi, theta) {
  z <- (mu - xi) / theta
  (1/theta) * exp(-z) * exp(-exp(-z))
}

m_y <- function(y, xi, theta, L = 1000) {
  integrand <- function(mu) f_y_given_mu(y, mu) * g_mu(mu, xi, theta)
  val <- integrate(integrand, lower = -L, upper = L,
                   rel.tol = 1e-10, abs.tol = 0)$value
  pmax(val, .Machine$double.xmin)
}

log_m_vec <- function(y, xi, theta, L = 1000) {
  vapply(y, function(yy) log(m_y(yy, xi, theta, L = L)), numeric(1))
}
```

```r
# ----------------------------
# One-sample MLE (xi, theta)
# Use par = (xi, logtheta) so theta>0 automatically
# ----------------------------
# ---- EV-aware starting values (moment-matching) ----
ev_start <- function(y) {
  gamma <- 0.5772156649015329
  v <- var(y)
  theta0 <- sqrt(pmax((v - 1) * 6 / pi^2, 1e-6))  # from Var(y)=pi^2/6*theta^2 + 1
  xi0 <- mean(y) - gamma * theta0                 # from E(y)=xi + gamma*theta
  c(xi0, log(theta0))
}

# ---- stable marginal density via z = (mu - xi)/theta ----
m_y_stable <- function(y, xi, theta, zL = -10, zU = 30) {

  integrand <- function(z) {
    # log integrand = log N(y | xi+theta z, 1) - z - exp(-z)
    log_int <- dnorm(y, mean = xi + theta * z, sd = 1, log = TRUE) - z - exp(-z)
    exp(log_int)
  }

  val <- integrate(integrand, lower = zL, upper = zU,
                   rel.tol = 1e-8, abs.tol = 0, subdivisions = 5000)$value
  pmax(val, .Machine$double.xmin)
}

log_m_vec_stable <- function(y, xi, theta, zL = -500, zU = 500) {
  vapply(y, function(yy) log(m_y_stable(yy, xi, theta, zL, zU)), numeric(1))
}

nll_one <- function(par, y, zL = -500, zU = 500) {
  xi <- par[1]
  theta <- exp(par[2])
  -sum(log_m_vec_stable(y, xi, theta, zL = zL, zU = zU))
}
```

```r
fit_one <- function(y, zL = -500, zU = 500) {
  start <- ev_start(y)

  opt <- optim(start, nll_one, y = y, zL = zL, zU = zU,
               method = "BFGS", hessian = TRUE,
               control = list(maxit = 2000))

  xi_hat <- opt$par[1]
  theta_hat <- exp(opt$par[2])

  V <- solve(opt$hessian)         # var on (xi, logtheta)
  se_xi <- sqrt(V[1, 1])

  list(
    xi_hat = xi_hat,
    theta_hat = theta_hat,
    logLik = -opt$value,
    se_xi = se_xi,
    ci_xi_wald = c(xi_hat - 1.96 * se_xi, xi_hat + 1.96 * se_xi),
    opt = opt, V = V,
    start = start,
    z_bounds = c(zL, zU)
  )
}
```

```r
fit50 <- fit_one(y50)
fit50
```

```
## $xi_hat
## [1] 4.392426
##
## $theta_hat
## [1] 2.052393
##
## $logLik
## [1] -120.1377
##
## $se_xi
## [1] 0.3433075
##
## $ci_xi_wald
## [1] 3.719543 5.065309
##
## $opt
## $opt$par
## [1] 4.3924260 0.7190063
##
## $opt$value
## [1] 120.1377
##
## $opt$counts
## function gradient
##       13        6
```

```
##
## $opt$convergence
## [1] 0
##
## $opt$message
## NULL
##
## $opt$hessian
##           [,1]      [,2]
## [1,]  8.587390 -2.615438
## [2,] -2.615438 66.575148
##
##
## $V
##             [,1]        [,2]
## [1,] 0.117860017 0.004630189
## [2,] 0.004630189 0.015202519
##
## $start
## [1] 4.3446355 0.7850768
##
## $z_bounds
## [1] -500  500
```

```r
fit50$ci_xi_wald
```

```
## [1] 3.719543 5.065309
```

```r
fit80 <- fit_one(y80, zL = -400, zU = 400)
fit80
```

```
## $xi_hat
## [1] 6.717204
##
## $theta_hat
## [1] 3.522375
##
## $logLik
## [1] -144.732
##
## $se_xi
## [1] 0.5448396
##
## $ci_xi_wald
## [1] 5.649318 7.785089
##
## $opt
## $opt$par
## [1] 6.717204 1.259135
##
## $opt$value
## [1] 144.732
##
```

```
## $opt$counts
## function gradient
##       13        6
##
## $opt$convergence
## [1] 0
##
## $opt$message
## NULL
##
## $opt$hessian
##           [,1]      [,2]
## [1,]  3.561864 -3.693987
## [2,] -3.693987 70.642980
##
##
## $V
##            [,1]       [,2]
## [1,] 0.29685020 0.01552257
## [2,] 0.01552257 0.01496738
##
## $start
## [1] 6.685512 1.320049
##
## $z_bounds
## [1] -400  400
```

```r
fit80$ci_xi_wald
```

```
## [1] 5.649318 7.785089
```

```r
# ------------------------------
# Profile-LR CI for xi (matches the "bisection/implicit function" hints)
# CI solves: 2(ell_hat - ell_prof(xi)) = qchisq(.95, df=1)
# ------------------------------
profile_logLik_xi <- function(xi_fixed, y, logtheta_center, zL = -10, zU = 30) {
  obj <- function(logtheta) nll_one(c(xi_fixed, logtheta), y = y, zL = zL, zU = zU)

  # constrain theta to a reasonable band around theta_hat
  # factor 20 band: [theta_hat/20, theta_hat*20]
  opt <- optimize(obj, interval = c(logtheta_center - log(20), logtheta_center + log(20)))

  list(logLik = -opt$objective, logtheta_hat = opt$minimum)
}

ci_xi_profile <- function(y, xi_hat, theta_hat, logLik_hat,
                          zL = -10, zU = 30, alpha = 0.05) {

  cutoff <- qchisq(1 - alpha, df = 1)

  target <- function(xi) {
    ell_prof <- profile_logLik_xi(xi_fixed = xi, y = y,
                                  logtheta_center = log(theta_hat),
```

```
                                  zL = zL, zU = zU)$logLik
    2 * (logLik_hat - ell_prof) - cutoff
  }

  # bracket around xi_hat and expand until crossing
  step0 <- max(0.25, 0.5 * theta_hat)

  left <- xi_hat - step0
  step <- step0
  for (k in 1:80) {
    if (is.finite(target(left)) && target(left) > 0) { left <- left - step; step <- step * 1.2 } else b
  }

  right <- xi_hat + step0
  step <- step0
  for (k in 1:80) {
    if (is.finite(target(right)) && target(right) > 0) { right <- right + step; step <- step * 1.2 } el
  }

  lower <- uniroot(target, interval = c(left, xi_hat))$root
  upper <- uniroot(target, interval = c(xi_hat, right))$root
  c(lower, upper)
}
```

```
ci50_prof <- ci_xi_profile(y50, fit50$xi_hat, fit50$theta_hat, fit50$logLik, zL=-10, zU=10)
ci80_prof <- ci_xi_profile(y80, fit80$xi_hat, fit80$theta_hat, fit80$logLik, zL=-10, zU=10)

ci50_prof
ci80_prof
```

```
# ----------------------------
# Two-sample models for LRT
# Full: (xi50,theta50) and (xi80,theta80)
# Reduced: common (xi,theta)
# ----------------------------
nll_full <- function(par, y50, y80, zL = -100, zU = 100) {
  xi1 <- par[1]; th1 <- exp(par[2])
  xi2 <- par[3]; th2 <- exp(par[4])

  -sum(log_m_vec_stable(y50, xi1, th1, zL = zL, zU = zU)) -
   sum(log_m_vec_stable(y80, xi2, th2, zL = zL, zU = zU))
}

nll_reduced <- function(par, y50, y80, zL = -100, zU = 100) {
  xi <- par[1]; th <- exp(par[2])

  -sum(log_m_vec_stable(y50, xi, th, zL = zL, zU = zU)) -
   sum(log_m_vec_stable(y80, xi, th, zL = zL, zU = zU))
}

fit_full <- function(y50, y80, zL = -100, zU = 100) {
  start <- c(fit50$xi_hat, log(fit50$theta_hat),
             fit80$xi_hat, log(fit80$theta_hat))
```

```r
  opt <- optim(start, nll_full, y50 = y50, y80 = y80, zL = zL, zU = zU,
               method = "BFGS", hessian = TRUE,
               control = list(maxit = 2000))

  list(
    xi50_hat = opt$par[1], th50_hat = exp(opt$par[2]),
    xi80_hat = opt$par[3], th80_hat = exp(opt$par[4]),
    logLik = -opt$value,
    opt = opt
  )
}

# EV-aware start for reduced model (optional but recommended)
ev_start <- function(y) {
  gamma <- 0.5772156649015329
  v <- var(y)
  theta0 <- sqrt(pmax((v - 1) * 6 / pi^2, 1e-6))
  xi0 <- mean(y) - gamma * theta0
  c(xi0, log(theta0))
}

fit_reduced <- function(y50, y80, zL = -100, zU = 100) {
  yall <- c(y50, y80)
  start <- ev_start(yall)   # better than median/sd for this model

  opt <- optim(start, nll_reduced, y50 = y50, y80 = y80, zL = zL, zU = zU,
               method = "BFGS", hessian = TRUE,
               control = list(maxit = 2000))

  list(
    xi_hat = opt$par[1], th_hat = exp(opt$par[2]),
    logLik = -opt$value,
    opt = opt
  )
}
```

```r
full <- fit_full(y50, y80, zL = -100, zU = 100)
red  <- fit_reduced(y50, y80, zL = -100, zU = 100)

LR   <- 2 * (full$logLik - red$logLik)
pval <- 1 - pchisq(LR, df = 2)

c(LR = LR, df = 2, p_value = pval)
```

```
##           LR           df      p_value
## 1.857906e+01 2.000000e+00 9.238632e-05
```

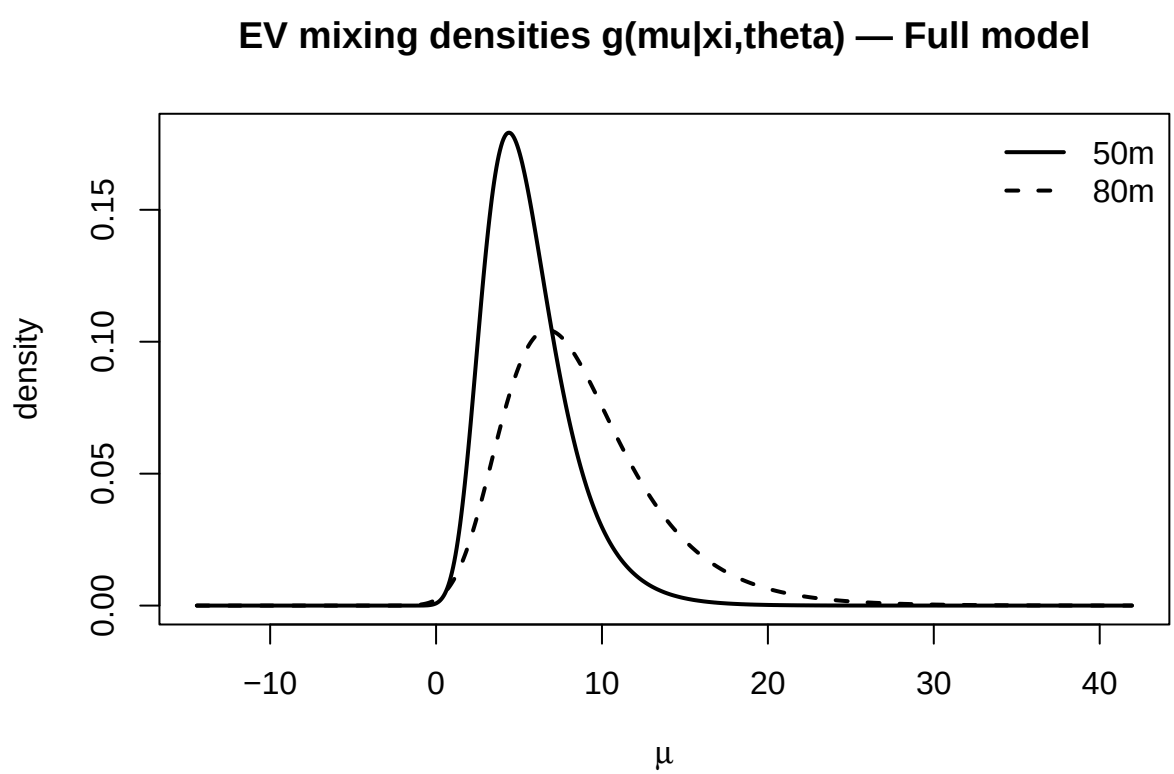
```r
# Overlay EV mixing densities (full model)
mu_grid <- seq(
  min(full$xi50_hat - 6*full$th50_hat, full$xi80_hat - 6*full$th80_hat),
  max(full$xi50_hat + 10*full$th50_hat, full$xi80_hat + 10*full$th80_hat),
  length.out = 800
```

```r
)

plot(mu_grid, g_mu(mu_grid, full$xi50_hat, full$th50_hat),
     type = "l", lwd = 2,
     xlab = expression(mu), ylab = "density",
     main = "EV mixing densities g(mu|xi,theta) - Full model")

lines(mu_grid, g_mu(mu_grid, full$xi80_hat, full$th80_hat),
      lwd = 2, lty = 2)

legend("topright",
       legend = c("50m", "80m"),
       lwd = 2, lty = c(1, 2), bty = "n")
```



Q2

In (2), $E(\mu) \approx \xi + 0.577\theta$, where 0.577 is Euler's constant (don't confuse with Euler's number). Using this expression, produce maximum likelihood estimate or estimates and 95% confidence interval or confidence intervals for the expected true maximum wind speed from your final model. Note that this will also be the expected observed maximum wind speed.

A:

```r
gamma <- 0.5772156649015329

wald_ci_mean_mu <- function(fit) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V      <- fit$V   # cov matrix of (xi, logtheta)

  eta_hat <- xi_hat + gamma * th_hat

  grad <- c(1, gamma * th_hat)          # d eta / d (xi, logtheta)
  se   <- sqrt(drop(t(grad) %*% V %*% grad))

  ci <- eta_hat + c(-1, 1) * qnorm(0.975) * se
  list(eta_hat = eta_hat, se = se, ci95 = ci)
}

ci50_eta_wald <- wald_ci_mean_mu(fit50)
ci80_eta_wald <- wald_ci_mean_mu(fit80)

ci50_eta_wald
```

```
## $eta_hat
## [1] 5.577099
##
## $se
## [1] 0.3875133
##
## $ci95
## [1] 4.817587 6.336611
```

```r
ci80_eta_wald
```

```
## $eta_hat
## [1] 8.750373
##
## $se
## [1] 0.6494937
##
## $ci95
## [1]  7.477389 10.023358
```

Q3

Find a maximum likelihood estimate or estimates (depending on your final model) of the median daily maximum wind speed or speeds. Compute the associated 95% approximate confidence interval or intervals.

A:

```
# ---------- FY(y | xi, theta) ----------
FY_stable <- function(y, xi, theta, zL = -10, zU = 30) {
  integrand <- function(z) {
    # weight = exp(-z - exp(-z))
    # inside = Phi(y - (xi + theta z))
    pnorm(y - (xi + theta * z)) * exp(-z - exp(-z))
  }
  val <- integrate(integrand, lower = zL, upper = zU,
                   rel.tol = 1e-8, abs.tol = 0, subdivisions = 5000)$value
  # numerical safety:
  pmin(pmax(val, 0), 1)
}

# ---------- median of Y given (xi, theta) ----------
median_y <- function(xi, theta, zL = -10, zU = 30) {
  f <- function(y) FY_stable(y, xi, theta, zL, zU) - 0.5

  # bracket around mean-ish scale; this is usually safe for wind speeds
  y0 <- xi + 0.5772156649015329 * theta  # uses Q2's mean approx as a center
  lo <- y0 - 10 * theta - 5
  hi <- y0 + 10 * theta + 5

  # expand if needed
  for (k in 1:60) {
    if (f(lo) > 0) lo <- lo - (2 + theta) else break
  }
  for (k in 1:60) {
    if (f(hi) < 0) hi <- hi + (2 + theta) else break
  }

  uniroot(f, interval = c(lo, hi))$root
}

# plug-in MLE medians
med50_hat <- median_y(fit50$xi_hat, fit50$theta_hat, zL = -100, zU = 100)
med80_hat <- median_y(fit80$xi_hat, fit80$theta_hat, zL = -100, zU = 100)

med50_hat
```

```
## [1] 5.207855
```

```
med80_hat
```

```
## [1] 8.04927
```

Q4

Find a maximum likelihood estimate or estimates (depending on your final model) for the probability that true daily maximum wind speed is between 4 and 17 m/s. Compute 95% approximate confidence intervals for the estimate or estimates.

A

```r
# EV (right-skew Gumbel) CDF for mu
G_mu <- function(u, xi, theta) {
  exp( -exp( -(u - xi)/theta ) )
}

p_between_mu <- function(fit, a = 4, b = 17) {
  xi <- fit$xi_hat
  th <- fit$theta_hat
  G_mu(b, xi, th) - G_mu(a, xi, th)
}

p50_hat <- p_between_mu(fit50, 4, 17)
p80_hat <- p_between_mu(fit80, 4, 17)

p50_hat
```

```
## [1] 0.6998669
```

```r
p80_hat
```

```
## [1] 0.8324576
```

Q5

What would you conclude about the question of whether there is or is not a benefit to placing turbine hubs at either 50 or 80 m in height?

A:

In Q1, the likelihood ratio test provided evidence that the wind speed distributions at 50m and 80m differ, leading us to select the full model with separate parameter estimates for each height. The estimated parameters imply systematically different wind speed behavior across heights.

In particular, the estimated probability that true daily maximum wind speed lies in the operationally relevant range of 4–17 m/s is larger at 80m than at 50m. The corresponding confidence intervals indicate that this difference is not attributable to sampling variability alone. Additionally, the estimated mean and median daily maximum wind speeds are higher at 80m.

Taken together, these results suggest that, under the fitted model and the conditions represented in the observed data, placing turbine hubs at 80m is associated with more favorable wind speed characteristics for turbine operation. This conclusion is conditional on the validity of the model assumptions and on future wind conditions being similar to those reflected in the data used for this analysis.

Q6

As an exercise in learning about statistical procedures, your friend would like to see a Monte Carlo approximation to the expected true wind speed for the two heights separately. Produce such approximations along with 95% Monte Carlo intervals. Your friend understands intuitively that these approximations and the maximum likelihood estimates of Question 2 should have the same or nearly the same values, but marvels at the fact that the intervals associated with these approximations are so much more narrow than those computed using the asymptotic theory of likelihood estimation in Question 2. Explain why this is the case and what it means.

A:

Monte Carlo

```r
set.seed(43)

gamma <- 0.5772156649015329

# Sample from right-skew Gumbel EV(xi, theta)
rEV <- function(n, xi, theta) {
  u <- runif(n)
  xi - theta * log(-log(u))
}

mc_E_mu <- function(fit, M = 200000) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat

  mu_draws <- rEV(M, xi_hat, th_hat)

  mc_mean <- mean(mu_draws)
  mc_se   <- sd(mu_draws) / sqrt(M)

  mc_ci <- mc_mean + c(-1, 1) * qnorm(0.975) * mc_se

  # For comparison: analytic plug-in
  plug_in <- xi_hat + gamma * th_hat

  list(
    M = M,
    plug_in = plug_in,
    mc_mean = mc_mean,
    mc_se = mc_se,
    mc_ci95 = mc_ci
  )
}

mc50 <- mc_E_mu(fit50, M = 200000)
mc80 <- mc_E_mu(fit80, M = 200000)

mc50
```

```
## $M
```

```
## [1] 2e+05
##
## $plug_in
## [1] 5.577099
##
## $mc_mean
## [1] 5.565262
##
## $mc_se
## [1] 0.005874134
##
## $mc_ci95
## [1] 5.553749 5.576775
```

```
mc80
```

```
## $M
## [1] 2e+05
##
## $plug_in
## [1] 8.750373
##
## $mc_mean
## [1] 8.763382
##
## $mc_se
## [1] 0.0101294
##
## $mc_ci95
## [1] 8.743529 8.783236
```

Comparison - Concise

The Monte Carlo (MC) intervals are much narrower because they measure only simulation (numerical) error conditional on the estimated parameters $(\hat{\xi}, \hat{\theta})$, whereas the asymptotic confidence intervals in Question 2 account for statistical uncertainty in the joint estimation of (ξ, θ) from finite data.

Comparison - More Details

For each height, we simulated

$$\mu^{(1)}, \ldots, \mu^{(M)} \sim \mathrm{EV}(\hat{\xi}, \hat{\theta})$$

and estimated the expected true wind speed by

$$\widehat{E}_{MC}(\mu) = \bar{\mu}_M = \frac{1}{M} \sum_{m=1}^{M} \mu^{(m)}.$$

By the Central Limit Theorem for simulation,

$$\bar{\mu}_M \approx \mathcal{N}\left(E(\mu), \frac{\mathrm{Var}(\mu)}{M}\right),$$

so an approximate 95% Monte Carlo interval is

$$\bar{\mu}_M \pm 1.96 \frac{s_M}{\sqrt{M}}.$$

The crucial point is that this interval treats $(\hat{\xi}, \hat{\theta})$ as fixed and quantifies only Monte Carlo approximation error, which decreases at rate $1/\sqrt{M}$ and can be made arbitrarily small by increasing M.

In contrast, the confidence interval in Question 2 is based on asymptotic likelihood theory for

$$E(\mu) = \xi + \gamma\theta, \qquad \gamma \approx 0.577,$$

and reflects uncertainty in estimating (ξ, θ) from $n = 50$ observations. That parameter uncertainty propagates into uncertainty about $E(\mu)$.

Thus, the Monte Carlo intervals are narrow because they measure numerical precision, not inferential uncertainty. The wider intervals from Question 2 are the appropriate measures of statistical uncertainty about the expected wind speed. To obtain a simulation-based interval comparable to Question 2, one would need to incorporate parameter uncertainty (e.g., via a parametric bootstrap), rather than conditioning on fixed MLEs.

Appendix